GECG10069 (561084) F26: Introduction to Programming (C++)

Lab 8 : Arrays



What you will learn from Lab 8

In this laboratory, you will learn how to use arrays.

TASK 8-1 : ARRAY AS ARGUMENT

✓ *Sorting* arranges the elements of an array in ascending or descending order.

```
#include <iostream>
using namespace std;

void DisplayArray(int [],int);
void InsertionSort(int [],int);

int main() {
    const int MaxSize = 10;
    int array[MaxSize] = {21,15,12,24,9,30,27,6,3,18};
    DisplayArray(array,MaxSize);
    InsertionSort(array,MaxSize);
    DisplayArray(array,MaxSize);
}

void DisplayArray(int vec[], int size) {
    for (int idx = 0; idx < size; idx++)
        cout << vec[idx] << " ";
    cout << endl;
}

void InsertionSort(int vec[],int size) {
    for (int jdx = 1; jdx < size; jdx++) {
        int key = vec[jdx];
        int idx = jdx - 1;

        while ( idx >= 0 && vec[idx] > key) {
            vec[idx+1] = vec[idx];
            --idx;
        }

        vec[idx+1] = key;
    }
}
```

TASK 8-2 : 2D ARRAY

```
#include <iostream>
#include <iomanip>
using namespace std;

const int numrows = 3;
const int numcols = 4;
void showarray(int [][numcols]);
int main()
{

    int val[numrows][numcols]={ { 1, 2, 3, 4},
                                { 5, 6, 7, 8},
                                { 9,10,11,12} };
    cout << "Display the array elements" << endl;
    showarray(val);
    return 0;
}
void showarray(int array[][numcols]){
    for(int row = 0; row < numrows; row++){
        for(int col = 0; col < numcols; col++){
            cout << setw(5)<< array[row][col] << " ";
       }
       cout << endl;
   }
}
```

EXERCISE 8-1: INNER PRODUCT

- ✓ Given two $m \times 1$ integer vectors A and B, please write a C++ program to compute their inner product.
- ✓ The inner product of A and B is:

$$A \cdot B = \sum_{i=1}^{m} a_i b_i = a_1 b_1 + a_2 b_2 + \cdots + a_m b_m$$

- ✓ where a_i and b_i represent the i-th elements of A and B, respectively.

Input

- ✓ The first line contains an integer m, representing the dimension of the two vectors.
  - ➢ $1 \leq m \leq 20$
- ✓ The second and third lines each contain m integers, representing the elements of A and B, respectively
  - ➢ $-100 \leq a_i, b_i \leq 100$

Output

- ✓ Output a single integer representing the computed inner product.
- ✓ For NYCU students: For this lab, you must strictly follow the specified output format. Do not print any additional prompts, instructions, or unnecessary messages, as they may interfere with the automated grading script and result in incorrect evaluation.

Submission (For NYCU students)

- ✓ Please submit your solution code to Exercise 8-1 to NTU COOL using the following filename format:
  - ➢ [student_id]_ex8-1.cpp
  - ➢ Example: 315511019_ex8-1.cpp
- ✓ Please strictly follow the required submission format and filename convention. If an incorrect filename or submission format prevents our automated grading script from compiling, executing, or evaluating your program correctly, you will be required to complete a make-up demo, and your score will be calculated according to the make-up submission grading policy.

Requirement

- ✓ Do not use any STL containers, such as `std::vector`, `std::array` or `std::list`.

Sample Input 1

5 1 2 3 4 5 6 7 8 9 10
Sample Output 1
130
Sample Input 2
4 1 -2 3 -4 5 6 -7 8
Sample Output 2
-60

<Hint>

- ✓ You may define a constant to specify the size of an array.
  - ➢ `const int MAX_SIZE = 20;`

EXERCISE 8-2: TETRIS

- ✓ Given a **10 × 10** Tetris playfield, your task is to output the playfield after performing all possible line clears.
- ✓ Each cell in the playfield is represented by one character
  - ➢ '**.**' represents an empty cell.
  - ➢ '#' represents an occupied cell.
- ✓ In Tetris, a horizontal line is full when all cells in that line are occupied by blocks.
- ✓ When a full line is cleared, the line disappears, and all rows above it **move down by one row** to fill the empty space. If multiple lines are full, all full lines are cleared, and the remaining rows move down accordingly.
- ✓ Empty lines are added at the top so that the playfield remains 10 rows by 10 columns.

Input

- ✓ The input contains **10 lines**.
- ✓ Each line contains **10 characters**, representing one row of the Tetris playfield from top to bottom.
  - ➢ Each character is either '**.**' or '#'

Output

- ✓ Output the Tetris playfield after all full lines have been cleared.
- ✓ Print exactly **10 lines**, with each line containing exactly **10 characters**.
- ✓ For NYCU students: For this lab, you must strictly follow the specified output format. Do not print any additional prompts, instructions, or unnecessary messages, as they may interfere with the automated grading script and result in incorrect evaluation.

Submission (For NYCU students)

- ✓ Please submit your solution code to Exercise 8-2 to NTU COOL using the following filename format:
  - ➢ [student_id]_ex8-2.cpp
  - ➢ Example: 315511019_ex8-2.cpp
- ✓ Please strictly follow the required submission format and filename convention. If an incorrect filename or submission format prevents our automated grading script from compiling, executing, or evaluating your program correctly, you will be required to complete a make-up demo, and your score will be calculated according to the make-up submission grading policy.

Requirement

✓ Do **not** use any STL containers, such as `std::vector`, `std::array` or `std::list`.

Sample Input 1

```
..........
..........
..........
..........
..........
..........
....##....
...####...
..######..
##########
```

Sample Output 1

```
..........
..........
..........
..........
..........
..........
..........
....##....
...####...
..######..
```

Sample Input 2

```
..........
..........
..........
..........
..##..##..
.###..###.
####..####
#####.####
##########
##########
```

Sample Output 2

```
..........
..........
..........
..........
..........
```

```
..........
..##..##..
.###..###.
####..####
#####.####
```

Sample Input 3

```
..........
..........
..........
..........
#########.
.#########
####.#####
#####.####
###.######
######.###
```

Sample Output 3

```
..........
..........
..........
..........
#########.
.#########
####.#####
#####.####
###.######
######.###
```

Sample Input 4

```
##########
....##....
...####...
##########
..######..
.########.
##########
###....###
####..####
##########
```

Sample Output 4

```
..........
..........
..........
```

```
..........
....##....
...####...
..######..
.########.
###....###
####..####
```

Sample Code

```
#include <iostream>
using namespace std;

void CreateInitialPlayfield(char[10][10]);
bool isFull(const char[10]);
void PrintPlayfield(const char[10][10]);

int main() {

  char playfield[10][10];

  CreateInitialPlayfield(playfield);

  if (isFull(playfield[0])) {
    for (int i=0; i<10; ++i) {
      playfield[0][i] = '.';
    }
  }

  for (int i=1; i<10; ++i) {
    if (isFull(playfield[i])) {
      for (int j=i-1; j>=0; --j) {
        for (int k=0; k<10; ++k) {
          playfield[j+1][k] = playfield[j][k];
        }
      }
      for (int j=0; j<10; ++j) {
        playfield[0][j] = '.';
      }
    }
  }

  PrintPlayfield(playfield);

  return 0;
}

// The function reads given playfield from input.
void CreateInitialPlayfield(char playfield[10][10]) {
  // to be implemented
  for (…) {
    for (…) {
      cin >> playfield[…][…];
    }
  }
}
```

```
// The function checks whether the line of playfield passed into it is full.
bool isFull(const char line[10]) {
  // to be implemented
}

// The function displays the playfield passed into it.
void PrintPlayfield(const char playfield[10][10]) {
  // to be implemented
}
```

<Hint>

- ✓ You may use a two-dimensional `char` array and declare it as `char playfield[10][10]` to store the Tetris playfield.
  - ➢ If you do so, it is recommended that you read and print the playfield one cell at a time using nested loops, rather than processing an entire row with operations like `cin >> playfield[i];`, `cout << playfield[i]` or `cin.getline(playfield[i], 10)`. The reason will be discussed in the following lecture.
- ✓ If you clear a full line as soon as you find it, checking the playfield from top to bottom may make the implementation easier.
- ✓ You may reference the given sample code. Most of the main logic has already been implemented in it. But there are three simple functions left to be finished
  - ➢ `CreateInitialPlayfield(), isFull(), PrintPlayfield()`